

Como o Transformer funciona

Uma explicação em alto nível, com um modelo treinado do zero

Resumo

O Transformer, apresentado em *Attention Is All You Need* (Vaswani et al., 2017), substituiu as redes recorrentes em tradução automática e virou a base de praticamente todos os modelos de linguagem atuais. Este relatório explica como ele funciona, do texto de entrada à frase traduzida, sem entrar no código. Todas as figuras de resultado vêm de uma implementação do zero em PyTorch, treinada no conjunto Multi30k (alemão→inglês) na configuração *base* do artigo: $N = 6$ camadas, $d_{\text{model}} = 512$, $h = 8$ cabeças, $d_{\text{ff}} = 2048$, 49,1 milhões de parâmetros. Os números mostrados são desse modelo, não ilustrações.

1 O problema que ele resolve

Traduzir uma frase exige olhar para ela inteira. Em “Ein Mann geht”, para escolher o verbo em inglês é preciso saber quem é o sujeito. Antes de 2017 isso era feito com redes recorrentes, que leem a frase palavra por palavra e carregam um estado de memória. Duas consequências ruins: a leitura é **sequencial**, então não dá para paralelizar o treino; e a informação entre duas palavras distantes precisa atravessar todos os passos intermediários, o que a degrada.

A proposta do Transformer é abandonar a recorrência e deixar cada posição olhar diretamente para todas as outras, de uma vez. O caminho entre duas palavras quaisquer passa a ter comprimento 1, e a frase inteira é processada em paralelo. O mecanismo que faz isso é a **atenção**, e é a única peça realmente nova da arquitetura.

2 A arquitetura em uma figura

O modelo tem duas metades (Figura 1). O **encoder** lê a frase de origem e produz uma representação dela; o **decoder** gera a tradução, um pedaço por vez, consultando essa representação. Cada metade é uma pilha de $N = 6$ camadas idênticas em estrutura, mas com pesos independentes.

Vale reparar em três coisas nessa figura, porque elas organizam todo o resto:

- A entrada de cada metade é a soma de um *embedding* com uma *codificação posicional*. Sem essa soma o modelo não teria noção de ordem, já que a atenção trata a frase como um conjunto.
- Cada bloco cinza tem sempre o mesmo formato: um sublayer (atenção ou rede feed-forward), seguido de *Add & Norm*. Isso se repete $2N$ vezes no encoder e $3N$ no decoder.
- O decoder tem um sublayer a mais que o encoder: a *cross-attention*, a seta que sai do encoder e entra no meio do decoder. É por ali que a tradução consulta a frase original.

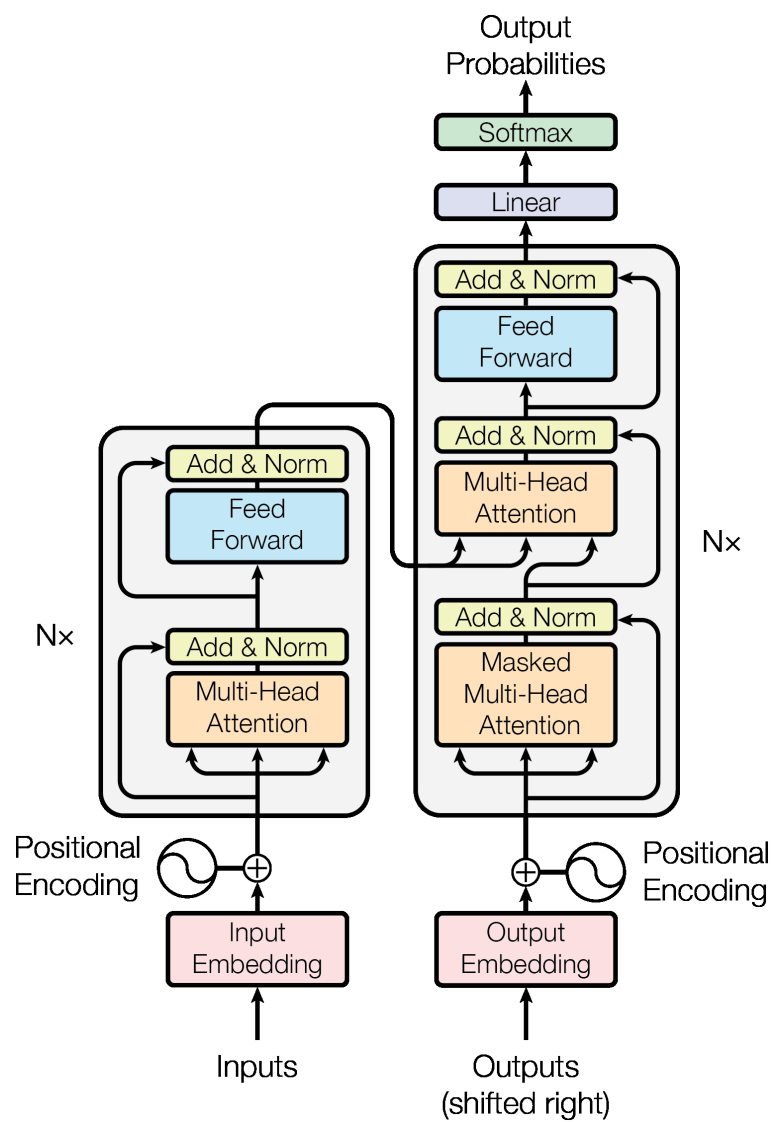


Figura 1: A arquitetura do artigo. Encoder à esquerda, decoder à direita, cada um repetido N vezes. Reproduzida da Figura 1 de Vaswani et al. (2017).

3 Do texto para números

3.1 Quebrar em pedaços

O modelo não trabalha com palavras inteiras, e sim com pedaços de palavra aprendidos estatisticamente, uma técnica chamada **BPE** (*byte-pair encoding*). Antes do treino, o algoritmo percorre o corpus e funde repetidamente o par de símbolos adjacentes mais frequente. Palavras comuns acabam virando um símbolo só; palavras raras permanecem quebradas em pedaços conhecidos (Figura 2).

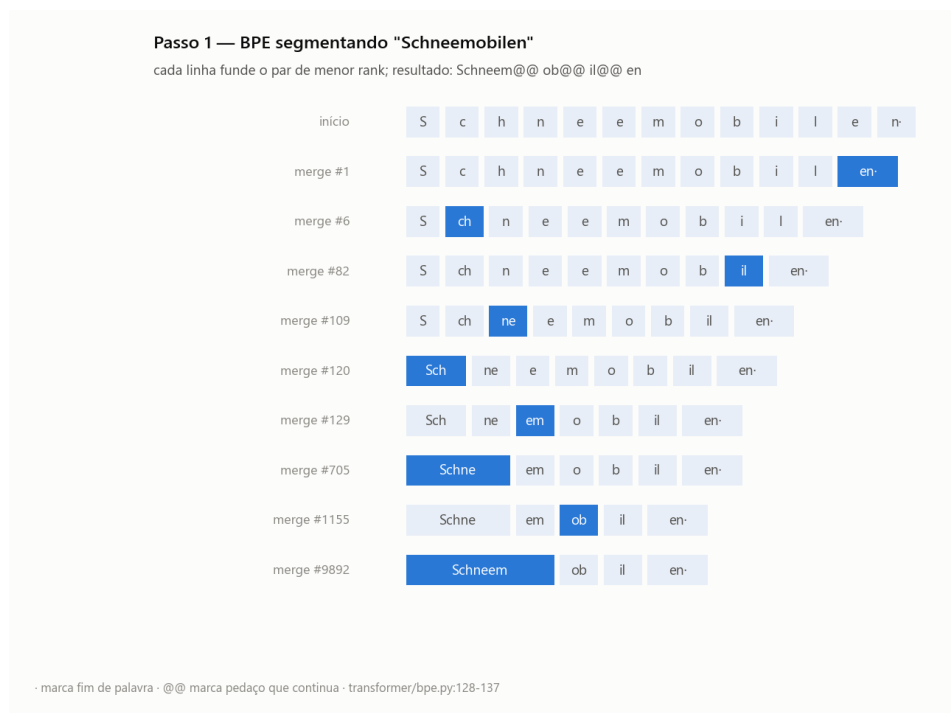


Figura 2: O BPE segmentando uma palavra rara. Cada linha aplica a fusão de menor *rank*, isto é, a que foi aprendida mais cedo por ser mais frequente. A primeira fusão é o sufixo alemão “-en”.

A vantagem é não ter palavras desconhecidas: qualquer sequência de caracteres pode ser montada a partir de pedaços menores. No nosso modelo, um vocabulário de 9 740 pedaços cobre alemão e inglês juntos, e nas 300 frases de teste não apareceu um único símbolo desconhecido.

Usar um vocabulário **compartilhado** entre as duas línguas não é detalhe: é o que permite amarrar três matrizes que, de outra forma, seriam separadas — o embedding da origem, o do destino e a projeção final antes do softmax passam a ser o mesmo tensor. Isso economiza parâmetros e faz o modelo tratar de forma consistente um pedaço que aparece nas duas línguas.

3.2 Embedding e escala

Cada pedaço vira um vetor de $d_{\text{model}} = 512$ números, buscado numa tabela que é aprendida no treino. Esse vetor é então multiplicado por $\sqrt{d_{\text{model}}} \approx 22,6$. A razão é a soma que vem a seguir: a codificação posicional tem valores entre -1 e 1 , e um vetor de posição tem norma 16. Sem a multiplicação, o embedding teria norma perto de 1 e seria abafado pela posição (Figura 3).

3.3 Onde entra a posição

Como a atenção não distingue ordem, a posição precisa ser injetada explicitamente. O artigo usa uma função fixa, sem parâmetros aprendidos: senos e cossenos de frequências que decaem

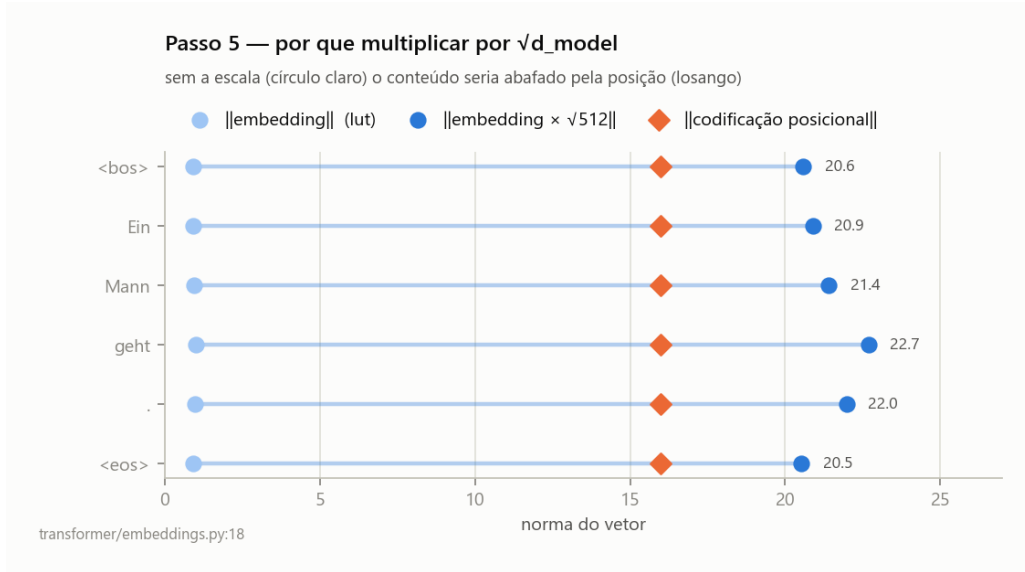


Figura 3: A norma de cada pedaço antes e depois da multiplicação por $\sqrt{d_{\text{model}}}$, com a norma da codificação posicional (losango) como referência. Depois da escala é o conteúdo que domina, e a posição entra como ajuste.

geometricamente ao longo das dimensões,

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (1)$$

O resultado (Figura 4) é que cada posição recebe uma assinatura única: as dimensões baixas oscilam rápido e distinguem posições vizinhas, as altas oscilam devagar e distinguem regiões da frase. Como é uma função e não uma tabela, ela vale para sequências mais longas que as vistas no treino — foi essa a justificativa do artigo para preferi-la a posições aprendidas, que dão resultado praticamente igual mas não extrapolam.

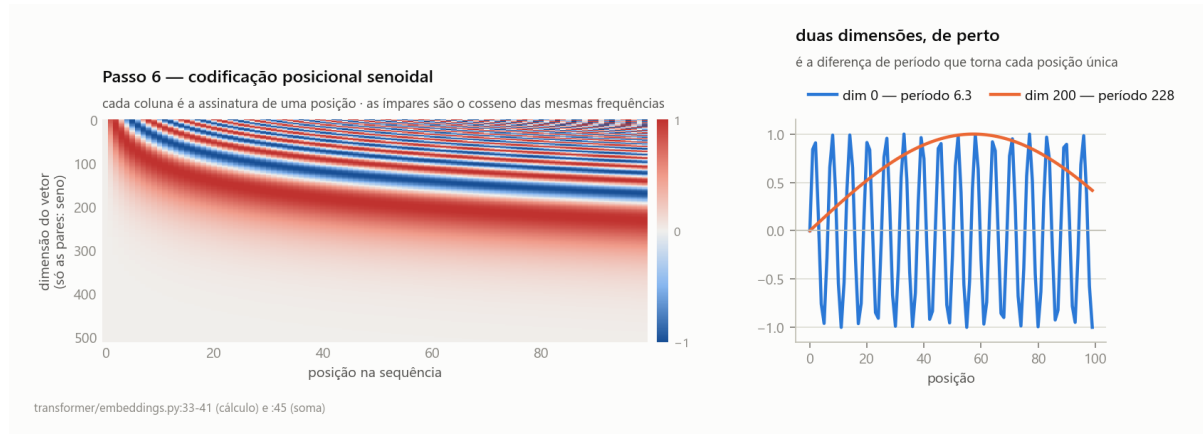


Figura 4: À esquerda, as 100 primeiras posições do vetor posicional. À direita, duas dimensões vistas de perto: a dimensão 0 tem período 6,3 e a 200 tem período 228. É a combinação de escalas que torna cada posição única.

4 Atenção: o mecanismo central

4.1 A ideia

Para cada posição, a atenção responde a uma pergunta: *de quais outras posições eu preciso, e o quanto?* Isso é feito com três projeções lineares do mesmo vetor de entrada, com nomes emprestados de bancos de dados:

- **Query** (Q) — o que esta posição está procurando.
- **Key** (K) — o que cada posição tem a oferecer.
- **Value** (V) — a informação que ela entrega se for escolhida.

O produto $QK^\top$ mede a compatibilidade entre cada query e cada key. Um softmax transforma esses números em pesos que somam 1, e a saída de cada posição é a média dos V ponderada por esses pesos:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2)$$

É neste ponto, e só neste, que a informação se move entre posições. Todo o resto da arquitetura processa cada posição isoladamente.

4.2 Por que dividir por $\sqrt{d_k}$

A divisão da Equação 2 é o “scaled” do nome e não é cosmética. O produto escalar de dois vetores de dimensão d_k tem variância proporcional a d_k ; com $d_k = 64$, os scores ficam grandes o bastante para saturar o softmax, que passa a dar peso ≈ 1 a uma posição e ≈ 0 a todas as outras. Nessa região o gradiente é praticamente nulo e o modelo para de aprender. A Figura 5 mostra os dois efeitos medidos no modelo treinado.

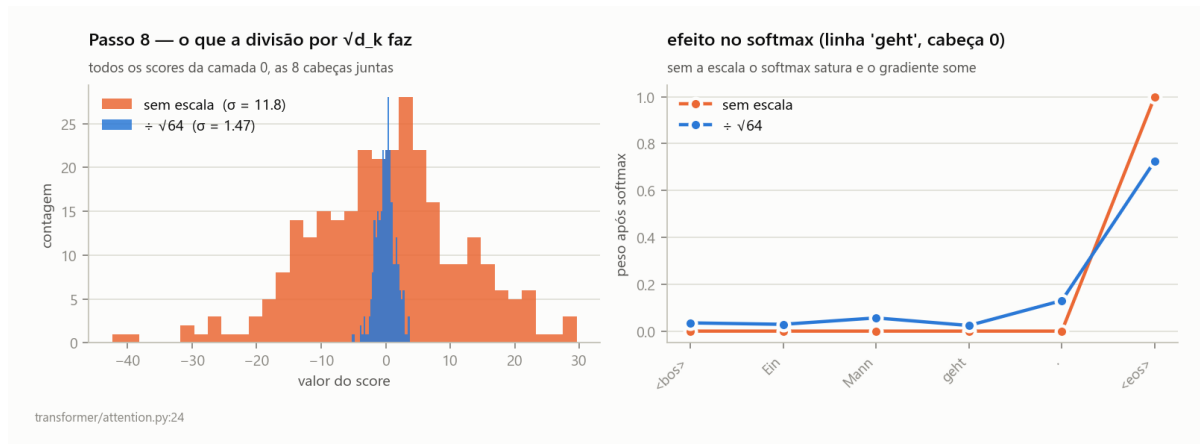
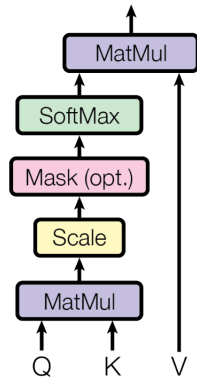


Figura 5: Efeito da divisão por $\sqrt{d_k}$. À esquerda, a distribuição de todos os scores de uma camada: o desvio padrão cai de 11,8 para 1,5. À direita, a consequência no softmax de uma linha: sem a escala ele colapsa em uma única posição.

4.3 Múltiplas cabeças

Em vez de uma atenção sobre vetores de 512 dimensões, o modelo faz $h = 8$ atenções em paralelo sobre subespaços de $d_k = 512/8 = 64$ dimensões, e concatena os resultados (Figura 6). O custo

Scaled Dot-Product Attention



Multi-Head Attention

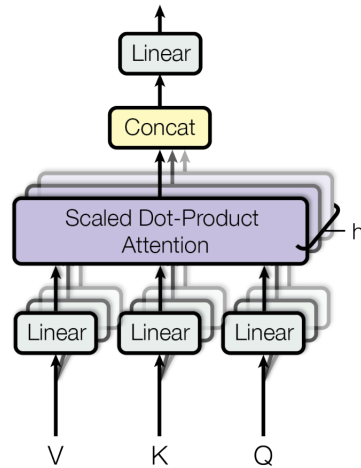


Figura 6: O mecanismo de atenção e sua versão multi-cabeça. Reproduzida da Figura 2 de Vaswani et al. (2017).

computacional é praticamente o mesmo, e o ganho é que cada cabeça pode se especializar em um tipo de relação diferente.

Que isso de fato acontece dá para verificar olhando as oito cabeças da mesma camada, na mesma frase (Figura 7): os padrões são visivelmente diferentes entre si.

4.4 Os três usos da atenção

A mesma operação aparece em três lugares, e o que muda é apenas de onde vêm Q , K e V :

Uso	Q , K , V vêm de	Para quê
Self-attention do encoder	todos da frase de origem	relacionar palavras da entrada
Self-attention do decoder	todos do que já foi gerado	manter a saída coerente
Cross-attention	Q do decoder; K, V do encoder	consultar a frase original

Tabela 1: Os três usos da atenção. A cross-attention é o único ponto de contato entre as duas metades do modelo.

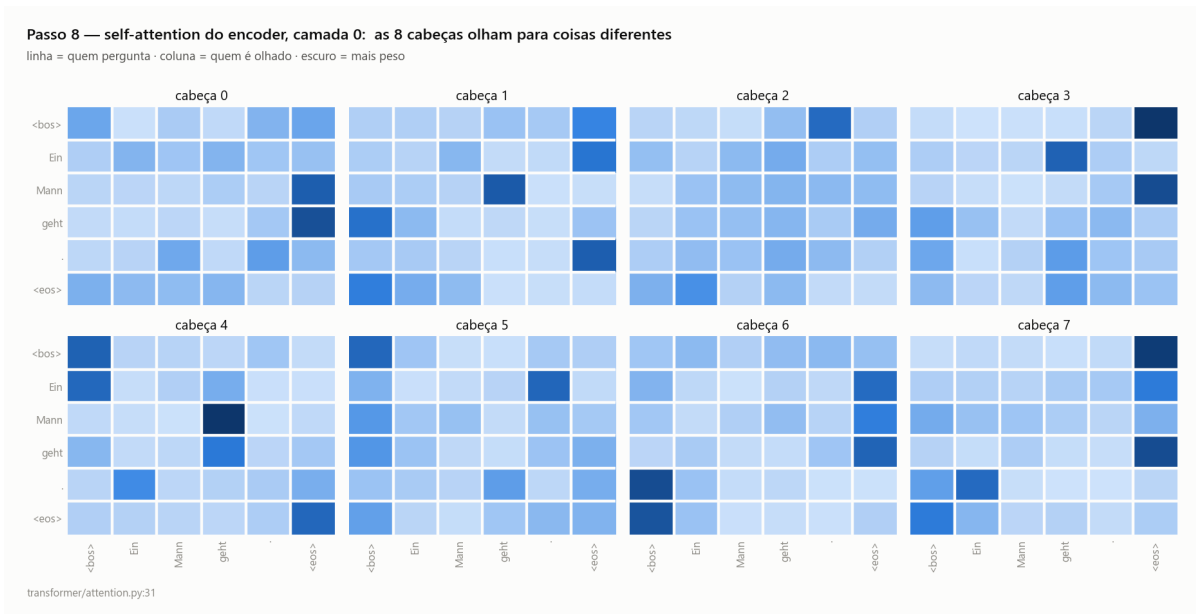


Figura 7: As oito cabeças da primeira camada do encoder, na frase “Ein Mann geht.”. Cada quadro é uma matriz de pesos: a linha é a posição que pergunta, a coluna é a que está sendo olhada.

5 O bloco que se repete

Em volta de cada sublayer há duas coisas que não mudam nunca:

- Uma **conexão residual**: a entrada é somada à saída, $x + \text{Sublayer}(x)$. Isso dá ao gradiente um caminho direto até as primeiras camadas e é o que torna viável empilhar seis blocos.
- Uma **normalização** (*layer normalization*) aplicada à soma, que devolve média 0 e desvio 1 em cada posição, mantendo os valores numa faixa estável ao longo da pilha.

O outro sublayer, além da atenção, é uma pequena rede densa aplicada *a cada posição separadamente*:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \quad (3)$$

que expande de 512 para 2048 dimensões, corta os negativos com ReLU e volta para 512. No modelo treinado, cerca de 93% das ativações intermediárias são zeradas pelo ReLU. A divisão de trabalho é limpa e vale memorizar: **a atenção mistura posições, a rede feed-forward processa cada posição**. Nenhuma outra parte da arquitetura move informação lateralmente.

6 Como a tradução é gerada

6.1 Uma palavra por vez, sem olhar o futuro

O decoder é auto-regressivo: gera um pedaço, acrescenta esse pedaço à sua própria entrada e repete. Durante o treino, porém, queremos prever todas as posições de uma vez, em paralelo — e aí surge um problema. Se o decoder puder ver a frase alvo inteira, prever a próxima palavra é trivial, basta copiá-la.

A solução é uma **máscara causal**: antes do softmax, as posições futuras recebem um valor muito negativo, e portanto peso zero (Figura 8). Cada posição só enxerga a si mesma e as anteriores. Com isso, as previsões de toda a frase podem ser calculadas em um único passe sem que nenhuma delas trapaceie.

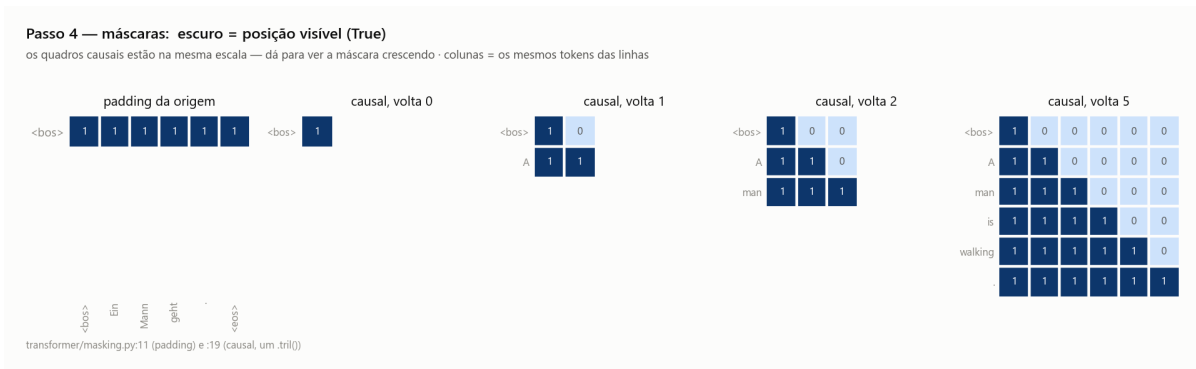


Figura 8: À esquerda, a máscara que marca quais posições da origem são texto de verdade e não preenchimento. À direita, a máscara causal do decoder crescendo a cada pedaço gerado: a posição i só enxerga $j \leq i$.

6.2 Escolhendo o próximo pedaço

A cada volta, a saída do decoder na última posição é projetada para o tamanho do vocabulário e passada por um softmax, virando uma distribuição de probabilidade sobre os 9 740 pedaços possíveis. A Figura 9 mostra as seis voltas que produziram a nossa tradução.

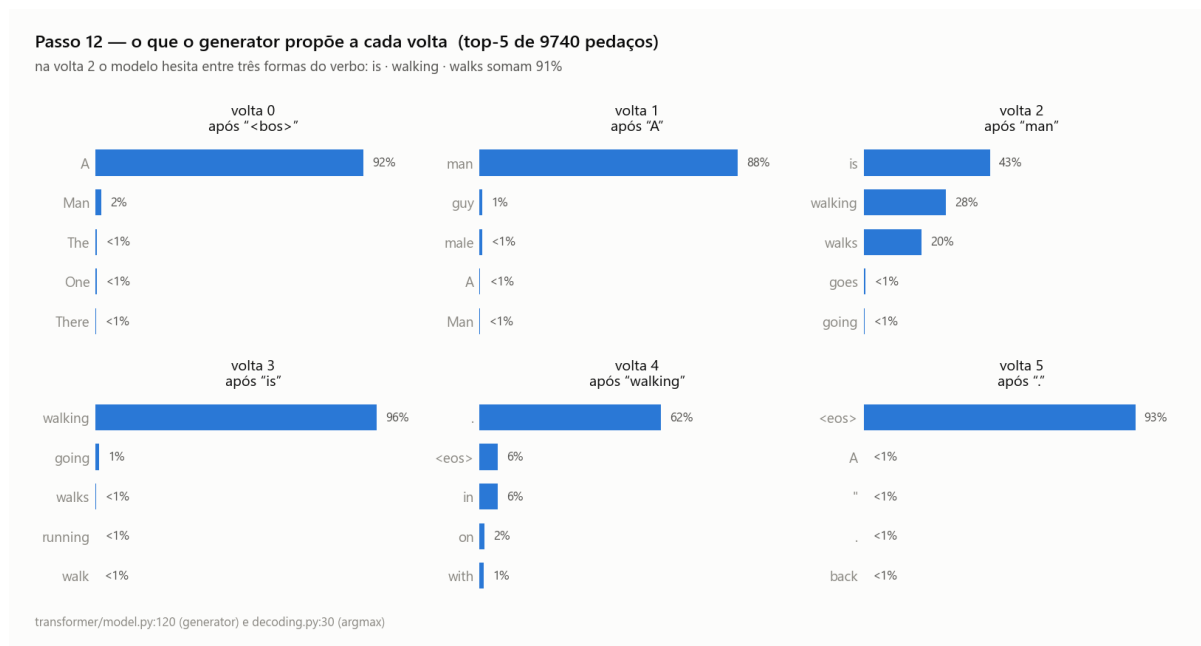


Figura 9: As cinco alternativas mais prováveis em cada volta. Na maioria delas o modelo está confiante; na volta 2 ele hesita entre três formas do verbo (*is*, *walking*, *walks*), que juntas somam 91% da massa.

Escolher sempre a alternativa mais provável (decodificação *greedy*) é simples mas míope: uma escolha ruim no início não pode ser desfeita. É por isso que se usa *beam search*.

6.3 Beam search: quem decide qual frase é a melhor

A resposta curta é que ninguém de fora decide. O juiz é o próprio modelo, e o critério é a probabilidade que ele atribui à frase inteira.

Como a probabilidade de uma frase é o produto das probabilidades dos pedaços, trabalha-se com o logaritmo, que transforma esse produto numa soma. O **score** de uma hipótese é a soma

dos $\log P$ de todos os pedaços que ela gerou até ali. Todo $\log P$ é negativo, então todo score é negativo e *menos negativo é melhor*.

O beam search mantém k hipóteses vivas ao mesmo tempo ($k = 4$ no artigo). A cada passo ele monta a matriz de todas as continuações possíveis — k feixes $\times$ 9 740 pedaços, quase 39 mil candidatos — e escolhe os k melhores **globalmente**. Essa palavra é a chave: não é cada feixe escolhendo o seu melhor pedaço. Um único feixe pode gerar vários dos sobreviventes, e outro pode não gerar nenhum e simplesmente desaparecer.

A Figura 10 mostra isso acontecendo na nossa frase. Repare no passo 2: **três dos quatro feixes descendem de “A man”**. Os ramos “guy” e “male”, que estavam vivos no passo 1, foram descartados — mesmo com uma boa continuação, o score acumulado deles não competia com as três alternativas que saíam de “A man”. É exatamente essa a diferença para o greedy, que teria fixado “is” no passo 2 e perdido “walking” e “walks” para sempre.

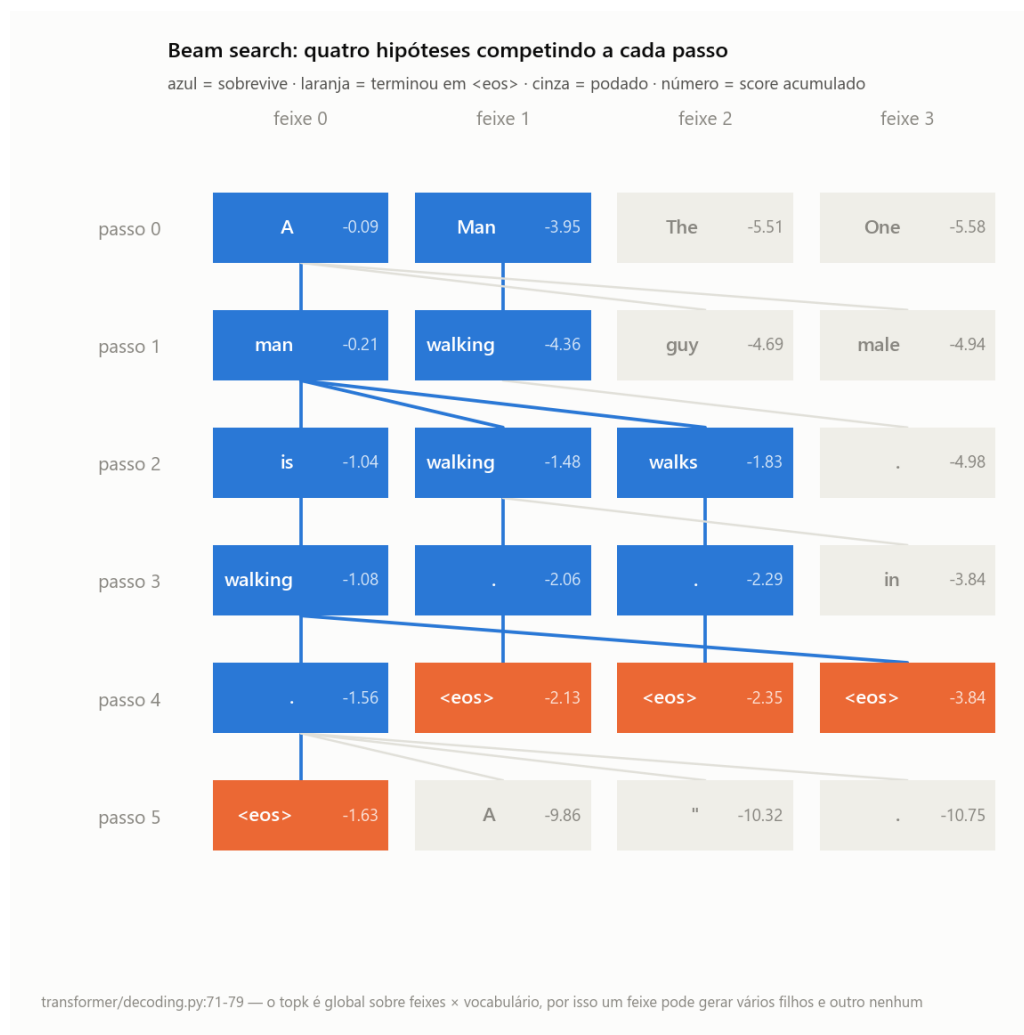


Figura 10: As quatro hipóteses vivas a cada passo, com o score acumulado. As linhas ligam cada hipótese ao feixe de onde ela saiu. No passo 2, três feixes descendem de “A man”; os ramos em cinza foram podados.

Quando um feixe gera <eos>, a frase está completa: ela é arquivada e sua vaga é liberada, para não continuar competindo. A busca termina quando há k frases completas.

A penalidade de comprimento. Falta uma correção, e é a única coisa aqui que não vem do modelo. Como o score é uma *soma* de números negativos, cada pedaço a mais piora o total:

sem intervenção, o modelo preferiria sistematicamente frases curtas e truncadas. A correção de Wu et al., usada no artigo, divide o score pelo comprimento amaciado por um expoente $\alpha = 0,6$:

$$\text{score final} = \frac{\text{score bruto}}{\left(\frac{5+|Y|}{6}\right)^\alpha}. \quad (4)$$

O ranqueamento final das quatro hipóteses da nossa frase:

#	final	bruto	Y	frase
1	-1,075	-1,630	7	A man is walking .
2	-1,482	-2,132	6	A man walking .
3	-1,635	-2,352	6	A man walks .
4	-2,672	-3,844	6	A man is walking

Tabela 2: As quatro frases completas, ordenadas pelo score final. A vencedora é a mais longa das quatro — é a penalidade que torna essa comparação justa.

Isso muda o resultado? Muda com frequência. Nas 60 primeiras frases do conjunto de teste, greedy e beam de 4 produziram traduções **diferentes em 26 delas**, ou 43%. Um exemplo:

DE Ein Mädchen in einem Karateanzug bricht ein Brett mit einem Tritt.

greedy A girl in a karate suit *is wearing* a board with a kick.

beam 4 A girl in a karate suit *takes* a board with a kick.

O greedy se comprometeu com “is” e daí “is wearing” era a continuação natural, que não faz sentido com “board”. O beam manteve o ramo “takes” vivo e ele venceu no acumulado.

Vale ser claro sobre o que o beam *não* faz: ele não avalia gramática, nem sentido, nem fidelidade ao alemão. Ele apenas procura melhor a frase que o modelo já considerava mais provável. Se o modelo estiver errado, o beam encontra o erro com mais eficiência — no exemplo acima, as duas versões ainda traduzem “bricht” (quebra) incorretamente. Beam search melhora a busca, não o modelo.

6.4 O alinhamento aparece sozinho

O resultado mais interessante não é a tradução em si, mas o que a cross-attention revela. Traçando, para cada pedaço gerado, onde o decoder estava olhando na frase alemã, obtém-se um alinhamento entre as duas línguas (Figura 11).

Ninguém forneceu esses alinhamentos. O modelo viu apenas pares de frases, e a correspondência palavra a palavra emergiu como subproduto de aprender a traduzir. É a evidência mais direta de que a atenção está fazendo o que se espera dela.

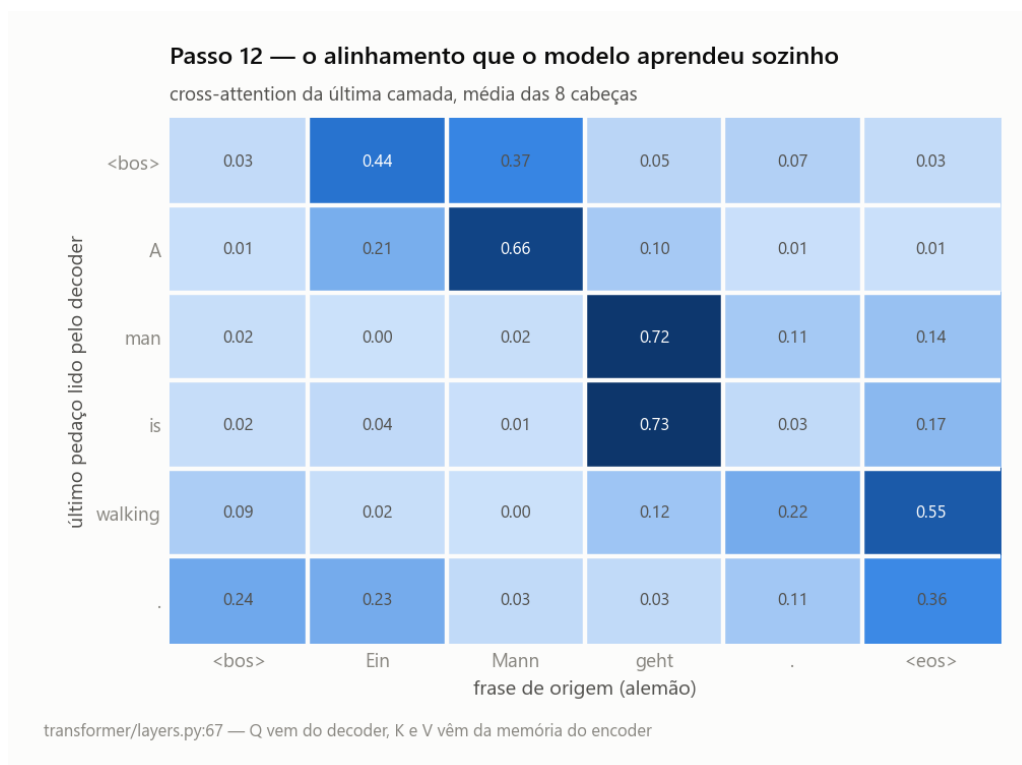


Figura 11: Alinhamento alemão→inglês extraído da cross-attention. Depois de ler “A”, o modelo olha para *Mann*; ao gerar “is” e “walking”, olha para *geht* nas duas voltas — um alinhamento de uma palavra alemã para duas inglesas.

7 Treinar o modelo

Três escolhas do artigo merecem menção, porque nenhuma delas é óbvia.

Taxa de aprendizado com aquecimento. Em vez de uma taxa fixa, o artigo usa um cronograma que sobe linearmente durante 4 000 passos e depois cai com $1/\sqrt{\text{passo}}$ (Figura 12):

$$lr = d_{\text{model}}^{-0.5} \cdot \min(\text{passo}^{-0.5}, \text{passo} \cdot \text{warmup}^{-1.5}). \quad (5)$$

O aquecimento não é opcional. Como a normalização é aplicada *depois* da soma residual, os gradientes das primeiras camadas são grandes no início do treino, e uma taxa alta desde o passo zero faz o modelo divergir.

Suavização de rótulos. Em vez de treinar o modelo para dar probabilidade 1 ao pedaço correto, reserva-se 10% da massa para os demais. Isso piora a perplexidade — o modelo fica deliberadamente menos confiante — mas melhora o BLEU, porque reduz a tendência a decorar.

Média de checkpoints. Ao final, os pesos das últimas gravações são promediados. É quase de graça e costuma valer alguns décimos de BLEU.

A Figura 13 mostra o treino do nosso modelo. A validação para de cair na época 13; o que fica salvo é a média dos cinco checkpoints até ali.

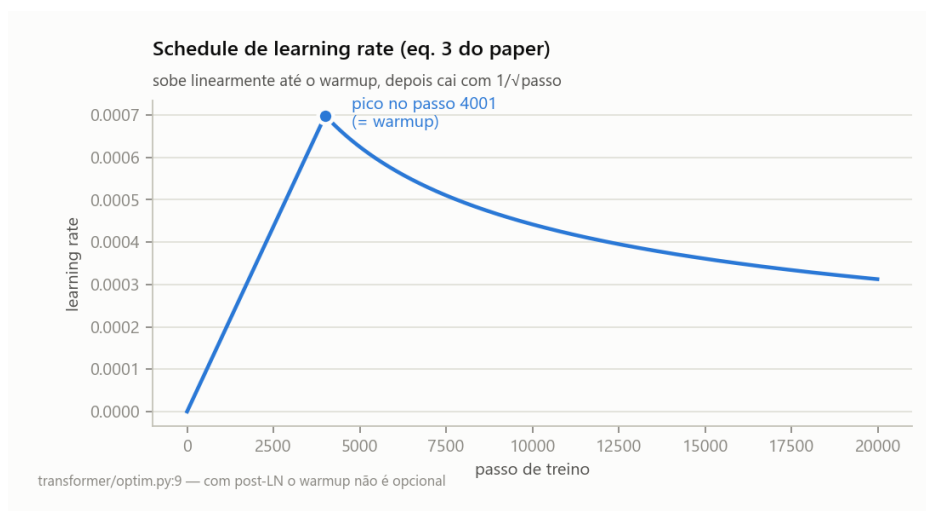


Figura 12: O cronograma de taxa de aprendizado, com o pico exatamente no fim do aquecimento.

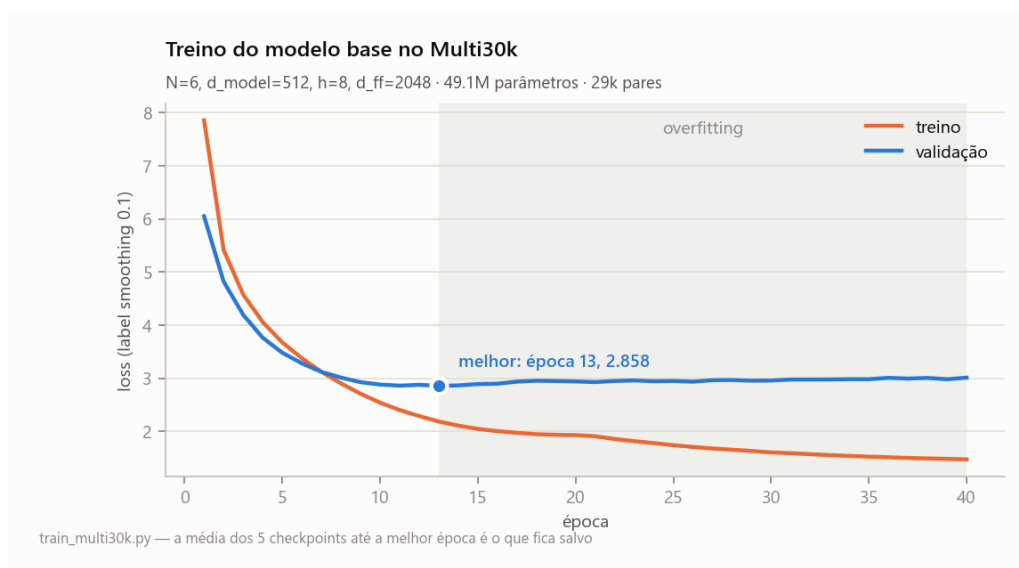


Figura 13: Treino no Multi30k. A partir da época 13 a perda de treino continua caindo enquanto a de validação sobe: o modelo passou a decorar.

8 Uma lição que o artigo não conta

Ao reproduzir a arquitetura fielmente, apareceu um problema que vale registrar, porque ele não está no artigo e é fácil de não perceber.

Cada camada mistura as posições pela atenção, e a normalização em seguida reescala o resultado. Repetindo isso seis vezes com a inicialização usual, as seis posições da saída do encoder já são quase o mesmo vetor *antes de qualquer treino*: a distância média entre elas cai para 18% da distância na entrada. Em um conjunto pequeno como o Multi30k, o treino converge então para um encoder que funciona como um “saco de palavras”, com cross-attention uniforme e sem nenhum alinhamento — e ainda assim traduz razoavelmente, o que torna a falha difícil de notar pela métrica.

A correção é multiplicar por $1/\sqrt{2N}$ as matrizes de saída de cada sublayer na inicialização, o que amortece o ramo residual. Com isso a distância na inicialização passa a 80%, e 96% no modelo treinado (Figura 14). O artigo não especifica a inicialização; esta foi a escolha que fez o modelo aprender alinhamentos em vez de médias.

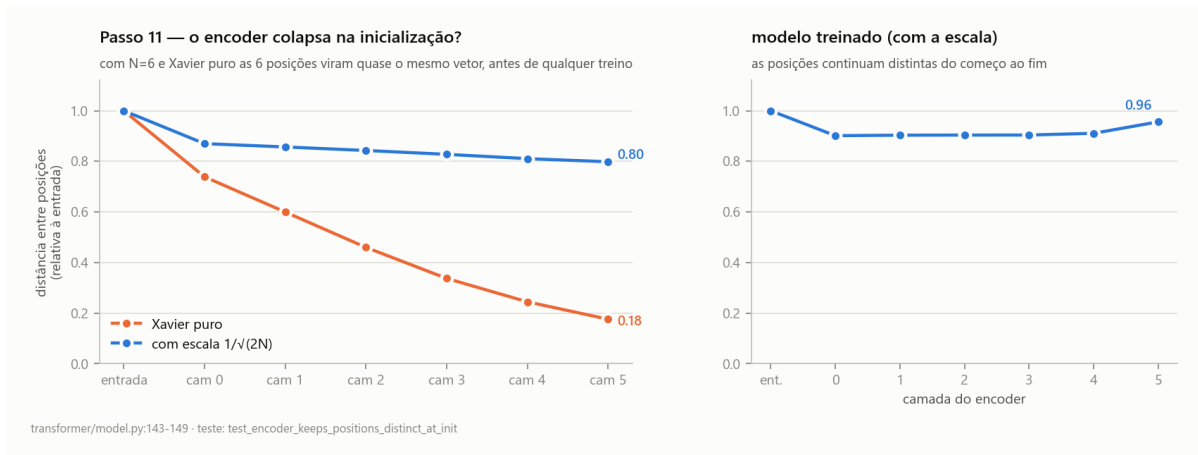


Figura 14: Distância média entre as posições ao longo das camadas do encoder, relativa à entrada. À esquerda, na inicialização, com e sem o amortecimento do ramo residual. À direita, o modelo treinado.

9 Resultados

Configuração	Valor
Camadas N (encoder e decoder)	6
Dimensão do modelo d_{model}	512
Cabeças de atenção h	8
Dimensão por cabeça $d_k = d_v$	64
Dimensão interna da FFN d_{ff}	2048
Dropout	0,1
Vocabulário BPE (compartilhado)	9 740
Parâmetros	49,1 M
Pares de treino (Multi30k)	29 000
Melhor época	13
BLEU-4 em 300 frases de teste	41,15

Tabela 3: Configuração e resultado do modelo treinado. A arquitetura é a *base* do artigo; o conjunto de dados é cerca de 150 vezes menor que o WMT14 usado nele.

Exemplos do conjunto de teste, nenhum visto durante o treino:

Alemão	Referência	Gerado
Leute Reparieren das Dach eines Hauses.	People are fixing the roof of a house.	People are fixing the roof of a house.
Ein Mann geht.	—	A man is walking.
Fünf Leute in Winterjacken und mit Helmen stehen im Schnee.	Five people wearing winter jackets and helmets stand in the snow.	Five people in winter jackets and helmets are standing in the snow.

Tabela 4: Traduções produzidas pelo modelo.

Para contexto: o artigo relata BLEU 27,3 em inglês→alemão no WMT14, uma tarefa muito mais difícil, com 4,5 milhões de pares e frases bem mais longas. Os dois números não são comparáveis; o valor aqui serve apenas para mostrar que a implementação aprende de fato.

10 Em resumo

1. O texto é quebrado em pedaços de palavra, cada pedaço vira um vetor, e a posição é somada a esse vetor.
2. A atenção deixa cada posição olhar diretamente para todas as outras. É a única operação que move informação lateralmente, e o caminho entre duas palavras quaisquer tem comprimento 1.
3. Múltiplas cabeças permitem que relações diferentes sejam capturadas em paralelo, em subespaços distintos.
4. Conexões residuais e normalização são o que tornam possível empilhar seis camadas dessas.
5. O encoder lê a frase uma vez; o decoder gera a tradução pedaço a pedaço, impedido de ver o futuro por uma máscara e consultando o encoder pela cross-attention.
6. O alinhamento entre as línguas não é programado: emerge do treino.

Nada disso é recorrente e nada é convolucional — daí o título do artigo.

Referências

- Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- Sennrich, R.; Haddow, B.; Birch, A. *Neural Machine Translation of Rare Words with Subword Units*. ACL, 2016. <https://arxiv.org/abs/1508.07909>
- Elliott, D. et al. *Multi30K: Multilingual English-German Image Descriptions*. 2016. <https://arxiv.org/abs/1605.00459>

As Figuras 1 e 6 são reproduzidas do artigo original; as demais foram geradas a partir do modelo treinado descrito neste relatório. O percurso do mesmo exemplo pelo código, linha a linha, está no README do repositório.